

Coding Assignments #7 (Part Three)

August 4, 2025

Directions:

1. You will be penalized for not following each of the directions exactly as written below. If you aren't sure about what something means, ask the professor!
 2. Put all your work in a single Haskell file.
 3. Submit your work (before the deadline) to Canvas.
 4. **Important:** You must avoid global helper functions. Keep the scope of your helper functions limited by using `lets` and `wheres`.
 5. **Important:** You will be graded on excessive use of helper functions when patterns or guards would make more sense.
 6. **Important:** You will lose points **each time** you use the `if` keyword. Use patterns and guards instead!
1. (Call your type class `Encodable`). Using only techniques from chapters 1-7, Make a type class that has two functions `encode` and `decode`. Your type class should take a type parameter `a`. The type of `encode` is `a -> Integer` and the type of `decode` is `Integer -> a`.
 2. Make an instance of `Encodable` for `Char`. The `encode` function is the `toInteger` function composed with the `fromEnum` function. (Recall that `fromEnum` takes a value (in this case `Char`) and turns it into an `Int`). The `decode` function does exactly the inverse operation: it is the `toEnum` function composed with the `fromInteger` function. (Recall that `toEnum` takes an `Integer` and turns it into a value (in this case of type `Char`)).
 3. Make an instance of `Encodable` for `[Char]`. (For this function to work, you may want to define an `Integer` called `base` to be `(maxBound :: Char)+1`). The `encode` function should work from the last character to the first. Starting with 0, the function should keep a running sum of the previous sum `* base` plus the `encode` of the previous `Char` in the `String`. Since

`base = 1114112`, the result of `encode "\0d"`, the string of the 0 character followed by the 100 character, is `111411200`, which is

$$100[= 'd'] * 1114112 + 0[= '\0'].$$

Notice how the string is encoded from the last character to the first. The `decode` character must be a precise inverse of `encode`.

4. make sure to `import qualified Text.Read`. It contains a function `readMaybe` that attempts to parse a value from a string. If successful, it computes `Just <something>`, where `<something>` is whatever was parsed. If unsuccessful, it computes `Nothing`. For example,

```
(Text.Read.readMaybe "3") :: Maybe Int
```

produces `Just 3`. You will need to be careful about types in this assignment, because Haskell cannot infer a type when parsing a value. Instead, you need to tell it explicitly how to do it.

5. (Call your function **`maybeSquareRoots`**. Using only techniques from chapters 1-7, write a function that takes in a list of strings and outputs a list of the same length. Anything in the list that can be parsed as a float should have `Just` its square root appear in the output list. Anything that can't be parsed as a float should have `Nothing` appear in its output list. So

```
maybeSquareRoots ["4", "pigeon", "25"]
```

should give you

```
[Just 2.0,Nothing,Just 5.0]
```